

Amazon SDE Interview Master Question Sheet

Curated Preparation Guide: DSA, System Design (LLD/HLD), Leadership Principles & The Bar Raiser

Target Roles: Amazon SDE-1 / SDE-2 (AWS & Consumer Tech)

Evaluation Pillars: Coding & DSA • System Design • 16 Leadership Principles (LPs)

Format: High-Yield Prep Sheet

1. AMAZON INTERVIEW STRUCTURE & EVALUATION FRAMEWORK

The Amazon Rule: Every single technical round allocates **15–20 minutes strictly to Leadership Principles (LPs)** using the **STAR method** (Situation, Task, Action, Result). Coding alone will not get you an offer; LP alignment and raising the bar across dimensions are equally mandatory.

Stage / Round	Technical Focus	LP & Behavioral Focus	Evaluation Benchmark
Online Assessment (OA) HACKERRANK	2 DSA Coding Questions (typically medium/hard leetcode: arrays, graphs, DP, sliding window). 70–90 mins.	Work Styles Assessment (scenarios prioritizing customer vs deadline) + Work Simulation.	100% test case pass rate on DSA. Consistency in leadership behavioral answers.
Technical 1 & 2 ONSITE / VIRTUAL LOOP	High-frequency DSA: Trees, Heaps/Priority Queues, Graphs, Two Pointers, Dynamic Programming.	<i>Customer Obsession, Bias for Action, Frugality, Ownership.</i>	Clean, production-grade bug-free code, modularity, edge-case analysis, Big-O proof.
LLD / System Design SDE-1 (LLD) / SDE-2 (HLD)	Object-Oriented Design (Locker System, Parking Lot, Vending Machine) or Distributed System (URL Shortener, E-Commerce Cart).	<i>Invent and Simplify, Are Right, A Lot, Think Big.</i>	Clean design patterns, extensible code, scalability, caching, DB indexing, microservices.
Bar Raiser Round CRUCIAL GATEKEEPER	Either challenging algorithmic problem or architecture review; cross-team interviewer.	Deep-dive interrogation into past failures, ethics, trade-offs, and <i>Earn Trust</i> .	Candidate must exceed the 50th percentile of current Amazon SDEs at the target level.

2. DSA: AMAZON HIGH-FREQUENCY CODING PROBLEMS

Amazon's coding rounds heavily emphasize data structures that power logistics, fulfillment centers, routing, and real-time carts: **Priority Queues, Graphs/BFS, Trees, and Interval Scheduling.**

A. Arrays, Intervals & Sliding Window

1. Merge Intervals / Non-overlapping Intervals

TOP AMAZON

Sort by start/end times and merge contiguous time slots. Crucial for Amazon delivery logistics and meeting schedulers. $O(N \log N)$.

2. Trapping Rain Water & Container With Most Water

Two-pointer approach vs Stack. Analyze optimal $O(N)$ time and $O(1)$ space without helper arrays.

3. Minimum Window Substring / Longest Substring K Distinct

Sliding window with character frequency hash map. Constant pointer expansion and contraction.

B. Heaps, Priority Queues & Strings

1. Top K Frequent Elements / K Closest Points to Origin

TOP AMAZON

Min-Heap of size K vs QuickSelect (Hoare's Selection) for $O(N)$ average time. Essential for fulfillment distance metrics.

2. Find Median from Data Stream (Two Heaps)

Max-heap for lower half, Min-heap for upper half. Balance logic for $O(1)$ median query and $O(\log N)$ insertion.

3. Merge K Sorted Lists / Arrays

Min-heap approach $O(N \log K)$ vs Divide & Conquer merge. Discuss pointer handling and memory locality.

4. Search in Rotated Sorted Array / Median of Two Sorted Arrays

Modified binary search with inflection points; binary search on partition sizes in $O(\log(\min(M, N)))$.

4. Reorganize String (Rearrange so no 2 adjacent same)

Max-heap tracking frequencies or greedy placement on even/odd indices. Handling impossible edge cases.

C. Trees, BST & Trie

1. Lowest Common Ancestor (LCA) in Binary Tree & BST

Recursive bottom-up traversal; handling scenarios where nodes might not exist in the tree.

2. Serialize and Deserialize Binary Tree **HARD**

Preorder traversal with delimiters and sentinel null markers or BFS queue encoding.

3. Binary Tree Maximum Path Sum & Diameter

Post-order recursion computing maximum single-branch contribution while updating global maximum.

4. Word Search II & Autocomplete System (Trie + DFS)

Trie prefix tree coupled with backtracking DFS over 2D grid with frequency priority tracking.

D. Graphs & Dynamic Programming

1. Number of Islands / Rotting Oranges (BFS/DFS)

TOP AMAZON

Multi-source BFS for propagation time (Rotting Oranges) and connected components in matrix.

2. Course Schedule I & II (Topological Sort / Kahn's Algorithm)

Detecting cycle in directed graph and ordering dependency graph using in-degree array & queue.

3. Word Ladder I & II (Shortest Transformation Sequence)

Bi-directional BFS optimization over intermediate generic state hashing (e.g., $d * g$).

4. Word Break / Coin Change / Longest Increasing Subsequence

Classic 1D/2D DP state transitions; bottom-up tabulation and binary search optimization for LIS ($O(N \log N)$).

3. SYSTEM DESIGN & OBJECT-ORIENTED DESIGN (OOD / LLD)

Area	Core Problems, Design Patterns & Architecture Topics
Low-Level Design (LLD / OOD) <i>Focus for SDE-1 & SDE-2</i>	• Design Amazon Locker Service: Locker sizes (S, M, L), package assignment, time-bound pin generation, locker availability tracking, state pattern for locker occupancy. • Design an In-Memory Key-Value Store with TTL / LRU Cache: Hash Map + Doubly Linked List for $O(1)$ get/put, background eviction thread or lazy expiration. • Design Parking Lot / Elevator System: Strategy pattern for parking slot selection, Observer pattern for floor requests, state machine for elevator transitions. • SOLID Principles & Design Patterns: Factory, Singleton, Strategy, Observer, Decorator, and Dependency Injection with production Java/C++/Python code snippets.
High-Level Design (HLD) <i>Focus for SDE-2</i>	• Design Amazon E-Commerce Order Fulfillment / Cart: Handling flash sale inventory depletion, distributed transactions (Saga Pattern vs 2PC), idempotency keys for checkout. • Design URL Shortener (TinyURL) / Rate Limiter: Token Bucket / Leaky Bucket algorithms, base62 encoding, Redis caching, Bloom filters for existence check. • Scalability & Distributed Storage: Consistent Hashing, Sharding strategies, Read-replicas, CAP theorem, Cache eviction (Write-through vs Write-back), Message Queues (Kafka / SQS).

4. AMAZON LEADERSHIP PRINCIPLES (LP) & STAR BEHAVIORAL FRAMEWORK

The STAR Formula: **Situation:** Set the context and magnitude (metrics, team size, timeline). **Task:** Your exact responsibility in the problem. **Action:** Specific technical and leadership steps *you* took (avoid "we"). **Result:** Quantified outcome (e.g., latency reduced by 35%, saved \$20k/month AWS cost).

Leadership Principle	Target Behavioral Questions	Key Winning Strategy & Pitfalls
Customer Obsession	• Tell me about a time you had to make a decision prioritizing the customer over short-term revenue or deadlines. • Give an example of anticipating a customer need before they asked.	Strategy: Start with the customer problem and work backwards. Pitfall: Focusing only on what pleased your manager rather than end users.
Ownership	• Describe a situation where you took on a task outside your official job description. • Tell me about a time you sacrificed short-term gains for long-term company health.	Strategy: Never say "that wasn't my job". Show cross-functional accountability and long-term thinking.
Bias for Action	• Tell me about a critical decision made with incomplete information or tight deadlines. • How do you balance speed vs quality when shipping features?	Strategy: Differentiate two-way doors (reversible decisions) vs one-way doors (irreversible). Value calculated risk-taking.
Have Backbone; Disagree and Commit	• Tell me about a time you disagreed with your manager or team lead on a technical approach. • How did you present your case, and what happened after the final decision?	Strategy: Show data-driven disagreement. Once the consensus was reached, commit 100% without passive resistance.
Earn Trust	• Describe a time you made a major mistake or caused an outage. How did you handle it? • Tell me about a time you received critical feedback you initially disagreed with.	Strategy: Take swift ownership without blaming others or pointing fingers. Detail the post-mortem and permanent fix.
Deliver Results	• Tell me about the most challenging project you delivered on a tight timeline. • How did you overcome roadblocks and ensure SLAs were met?	Strategy: Emphasize resilience, removing bottlenecks, prioritization, and measurable metrics of success.

5. LIVE CODING CHECKLIST & BAR RAISER SECRETS

Live Coding Execution Framework:

- **Clarify Inputs & Boundaries (0–5 min):** Ask about constraints: array bounds, duplicates, negative numbers, memory limits.
- **Brute Force & Trade-Offs (5–10 min):** State the naive solution, calculate Big-O, and suggest optimal data structure before typing.
- **Clean Modular Coding (10–25 min):** Write readable code with self-explanatory variable names; avoid monolithic methods.
- **Dry-Run with Edge Cases (25–35 min):** Walk through code manually with an example, empty input, 1-element, and maximum bounds.

Bar Raiser Evaluation Criteria:

- **Self-Correction:** Do you spot your own bugs when prompted, or do you get defensive?
- **Depth of Inquiry:** When asked "Why did you choose a HashMap over a TreeMap?", can you explain memory layout and hash collisions?
- **Culture Add:** Demonstrating humility, hunger to learn, and adherence to high technical standards.
- **Story Matrix:** Prepare at least 5 distinct project stories that can each adapt to 2–3 different LPs.